

Homework 2 – Theory

1

Perceptron begins at $w = (0, 0)$ and predicts $\text{sign}(w \cdot x)$. An example (x, y) counts as a mistake when

$$y(w \cdot x) \leq 0,$$

and every mistake updates the weights by $w \leftarrow w + yx$. The two runs below each make one pass through the data.

The answer to both questions is no, and three points are enough to show it. Let

$$x^1 = (2, 1), \quad y^1 = +1, \quad x^2 = (1, -1), \quad y^2 = -1, \quad x^3 = (0, 1), \quad y^3 = +1.$$

These are separable, since $w^* = (0, 1)$ gives $y^i(w^* \cdot x^i) = 1$ for each one.

Order $S = (x^1, x^2, x^3)$. The first example is always a mistake, because $w \cdot x^1 = 0$, and the update gives

$$w = (0, 0) + (2, 1) = (2, 1).$$

The second example has

$$w \cdot x^2 = 2(1) + 1(-1) = 1, \quad y^2 = -1,$$

so the prediction has the wrong sign and the weights move again:

$$w = (2, 1) - (1, -1) = (1, 2).$$

The third example has

$$w \cdot x^3 = 1(0) + 2(1) = 2, \quad y^3 = +1,$$

which is correct, so nothing changes. This run ends with

$S : \quad 2 \text{ mistakes}, \quad w = (1, 2).$

Order $S' = (x^3, x^1, x^2)$. The first example is again a mistake, since $w \cdot x^3 = 0$, and now

$$w = (0, 0) + (0, 1) = (0, 1).$$

The second example has

$$w \cdot x^1 = 0(2) + 1(1) = 1, \quad y^1 = +1,$$

which is correct, and the third has

$$w \cdot x^2 = 0(1) + 1(-1) = -1, \quad y^2 = -1,$$

which is also correct. This run ends with

$$\boxed{S' : \quad 1 \text{ mistake}, \quad w = (0, 1).}$$

The two orders give different numbers of mistakes and different final weights, and since $(1, 2)$ is not a multiple of $(0, 1)$, they describe different halfspaces as well. Both vectors already classify all three points correctly, so running until convergence would not change either result.

2

Let $z = y w^T x$, so that $z > 0$ means the example is classified correctly and $z < 0$ means it is a mistake. The loss

$$\phi(z) = \max(0, -z) = \begin{cases} 0, & z \geq 0, \\ -z, & z < 0, \end{cases}$$

is therefore zero on correct examples and equal to $|z|$ on mistakes, and its derivative is

$$\phi'(z) = \begin{cases} 0, & z > 0, \\ -1, & z < 0. \end{cases}$$

The problem says not to worry about $z = 0$, so treat it as a mistake and set $\phi'(0) = -1$. Perceptron does the same thing, updating on its first example even though $w \cdot x = 0$ there.

Since $\nabla_w (y^i w^T x^i) = y^i x^i$, the chain rule gives

$$\nabla_w \phi(y^i w^T x^i) = \begin{cases} 0, & y^i w^T x^i > 0, \\ -y^i x^i, & y^i w^T x^i < 0, \end{cases}$$

and substituting this into $w_{\text{new}} = w_{\text{old}} - \eta \nabla_w \phi$ leaves

$$w_{\text{new}} = \begin{cases} w_{\text{old}}, & \text{correct,} \\ w_{\text{old}} + \eta y^i x^i, & \text{mistake.} \end{cases}$$

Setting $\eta = 1$ turns this into

$$\boxed{w_{\text{new}} = \begin{cases} w_{\text{old}}, & \text{correct,} \\ w_{\text{old}} + y^i x^i, & \text{mistake,} \end{cases}}$$

which is the Perceptron update rule.

The step size does not matter either. Every update is multiplied by η and both algorithms start from the same weights, so the SGD weights remain η times the Perceptron weights throughout. Scaling by a positive constant does not change $\text{sign}(w \cdot x)$, so the two algorithms predict the same labels and make the same mistakes.

Perceptron is therefore SGD on $\frac{1}{m} \sum_i \phi(y^i w^T x^i)$, started at $w = 0$ with $\eta = 1$, taking the examples in order rather than at random.

3

(a)

The target $c = h_{\theta_1, \theta_2, b}$ divides the line into three pieces,

$$L = (-\infty, \theta_1), \quad M = [\theta_1, \theta_2], \quad R = (\theta_2, \infty),$$

where c equals $-b$ on L , b on M , and $-b$ again on R . Writing p_L, p_M, p_R for the probabilities of the three pieces,

$$p_L + p_M + p_R = 1.$$

A stump has a single threshold, so it splits the line in two. It can match c on two of the pieces but has to be wrong on the third, which leaves three choices:

- $h_{\theta_2, b}$ predicts b on $L \cup M$ and $-b$ on R , so it is wrong only on L and has error p_L .
- $h_{\theta_1, -b}$ predicts $-b$ on L and b on $M \cup R$, so it is wrong only on R and has error p_R .
- $h_{B, -b}$ predicts $-b$ across all of $[-B, B]$, so it is wrong only on M and has error p_M .

Three numbers that sum to 1 cannot all be larger than $\frac{1}{3}$, so

$$\min(p_L, p_M, p_R) \leq \frac{1}{3},$$

and taking the stump that matches the smallest of them gives

$$\text{some } h \in H \text{ satisfies } \Pr[h(x) \neq c(x)] \leq \frac{1}{3}.$$

(b)

Sort the sample so that $x_{(1)} \leq \dots \leq x_{(m)}$, with labels $y_{(1)}, \dots, y_{(m)}$. A stump predicts b' on the first k sorted points and $-b'$ on the rest, and any threshold between $x_{(k)}$ and $x_{(k+1)}$ gives the same predictions. Only the pair (k, b') matters, which leaves $2(m+1)$ candidates, each with error

$$\text{err}_S(k, b') = \#\{j \leq k : y_{(j)} = -b'\} + \#\{j > k : y_{(j)} = b'\}.$$

A single sweep finds all of these errors. At $k = 0$ every point is predicted $-b'$, so $\text{err}_S(0, b') = \#\{j : y_{(j)} = b'\}$, and raising k by one moves $x_{(k)}$ to the side predicted b' , which fixes that point if its label is b' and breaks it otherwise:

$$\text{err}_S(k, b') = \text{err}_S(k-1, b') + \begin{cases} -1, & y_{(k)} = b', \\ +1, & y_{(k)} = -b'. \end{cases}$$

The procedure is to sort the sample, sweep k from 0 to m for $b' = +1$ and $b' = -1$ while keeping the smallest error, and return the corresponding stump. Sorting costs $O(m \log m)$ and each sweep costs $O(m)$, for a total of

$$O(m \log m).$$

The result has the smallest training error of any stump, so it is an ERM.

(c)

A stump can label m points in only $2(m+1)$ ways, so there are very few hypotheses to consider, and the training error of each one stays close to its true error once m is reasonably large. The stump from part (b) therefore has true error close to $\frac{1}{3}$.

The reason this is easier than learning C is the accuracy being asked for. Learning C would mean driving the error down to ϵ and locating both θ_1 and θ_2 , so the sample size would have to grow as ϵ shrinks. A weak learner for H only needs a fixed advantage over $\frac{1}{2}$, which a fixed sample size already gives. Boosting can then take that advantage and drive the error down to ϵ , so C is weakly learnable using H .

4

At iteration t , each example x_i carries a weight w_i , the total weight is $W = \sum_i w_i$, and the distribution is $D_t(i) = w_i/W$. The hypothesis h_t returned at this iteration has

$$E_t = \frac{1}{W} \sum_{i: h_t(x_i) \neq y_i} w_i, \quad A_t = 1 - E_t, \quad \beta_t = \frac{E_t}{A_t},$$

and the reweighting multiplies the weight of each correct example by β_t while leaving the wrong ones as they are:

$$w_i^{\text{new}} = \begin{cases} \beta_t w_i, & h_t(x_i) = y_i, \\ w_i, & h_t(x_i) \neq y_i. \end{cases}$$

Assume $0 < E_t < 1$, so that β_t is defined and positive.

Before the update the correct examples hold $A_t W$ of the weight and the wrong ones hold $E_t W$. Scaling the correct side gives

$$\beta_t \cdot A_t W = \frac{E_t}{A_t} \cdot A_t W = E_t W,$$

which is exactly what the wrong side already holds, so the new total is

$$W^{\text{new}} = E_t W + E_t W = 2E_t W,$$

and the accuracy of h_t on the new distribution is

$$\boxed{\Pr_{i \sim D_{t+1}} [h_t(x_i) = y_i] = \frac{E_t W}{2E_t W} = \frac{1}{2}.$$

The factor β_t is chosen precisely so that the correct side shrinks until it matches the wrong side, and both E_t and W cancel, so the answer is exactly $\frac{1}{2}$ for any h_t and any set of weights. This leaves h_t no better than guessing on D_{t+1} , so the next weak learner has to get a different set of examples right.